inmind.academy
by inmind.ai

Gen AI Track | Spring 2026

Final Project

Production AI Agent System

REQUIREMENTS, RUBRIC, AND GUIDELINES

1. Overview

Build a production-ready multi-agent AI system that solves a real-world problem. Your system must demonstrate mastery of the full stack covered in this track: RAG, agents, multi-agent orchestration, tool calling, MCP, evaluation, API serving, and containerized deployment.

This is an individual project. No group work.

If multiple students choose the same topic, they will be evaluated competitively. The better implementation earns bonus recognition.

Project topics are provided in a separate document. Choose one topic or propose your own (requires instructor approval).

2. Architecture Requirements (Mandatory)

Your system MUST have all of the following components. These are non-negotiable.

2.1 Two Independent Agent Systems on Different Tech Stacks

You must build two separate agent systems that communicate with each other:

- **Agent System A:** Built with LangGraph (Python). This is your primary system with core business logic.
- **Agent System B:** Built with Google ADK OR another framework. This is an independent service that Agent System A can call.

Why two systems? In real-world production, AI systems do not exist in isolation. A travel agency agent calls a hotel booking agent run by a different company. A customer support agent calls a billing system built by a different team. Your project simulates this: two independent agent systems, each in its own Docker container.

Agent System B must NOT be a Python import inside Agent System A. They must communicate via network (HTTP requests between containers) or A2A.

2.2 RAG Pipeline

At least one agent system must include a RAG pipeline with:

- Document ingestion (at least 15-20 pages of domain-specific content)
- Chunking with a justified strategy (explain why you chose your chunk size and method)
- Embedding model (local or API, justify your choice)
- Vector database (Qdrant, available through docker)
- Retrieval with metadata filtering (Optional)

2.3 MCP Server

Build at least one MCP server that exposes tools or resources from your domain:

- Runs in its own Docker container
- Exposes at least 2 tools relevant to your domain

- Callable by one of your agent systems

2.4 Dockerized Microservice Architecture

Your entire system must run as separate Docker containers orchestrated with docker-compose:

Container	What It Runs
agent-system-a	LangGraph-based primary agent system + FastAPI
agent-system-b	Google ADK (or other) secondary agent system + FastAPI
mcp-server	Your custom MCP server
vector-db	Qdrant
mongo + redis (optional)	Session storage (if applicable)

Each container must have its own Dockerfile and be independently deployable. Agent System A calls Agent System B via HTTP and/or A2A, not by importing Python modules.

2.5 API Layer

Both agent systems must be exposed as FastAPI APIs with:

- POST endpoint for chat/query
- Streaming support for at least one endpoint
- Session management (conversation history persists across requests)
- Proper error handling and timeouts

2.6 Guardrails and Safety

- Input and Output guardrail on the primary
- Iteration limits on all agent loops
- Request timeouts

2.7 Evaluation

- Test set of at least 20 questions with ground truth answers and source references
- Retrieval metrics computed for Rag evaluation
- Generation evaluation: automated evaluation using RAGAS/LLM-as-judge (faithfulness, correctness, relevance) on all 20 questions
- At least 2 configuration comparisons (e.g., chunk size A vs B, top-K 3 vs 5, different embedding model) with metrics showing which is better
- 3 documented failure cases with root cause analysis (what went wrong and why)

2.8 Bonus: Voice Interface (up to 10 extra points)

If your hardware and budget allow:

- STT input (Whisper, local or API)

- TTS output (gTTS, OpenAI TTS, Piper)
- End-to-end voice conversation demo
- Add Bert / finetune Bert for a custom use case to be used as a tool by your agent (example: classify reviews incase of booking agent).

3. Evaluation Rubric (100 points + 10 bonus)

Category	Item	Points
Architecture (20)		
	Agent System A: LangGraph supervisor + 2+ specialists + tools	10
	Agent System B: different framework, independent service, own logic	5
	Inter-system communication: A calls B via HTTP API	5
RAG Pipeline (20)		
	Document quality: 15+ pages, domain-relevant, well-structured	5
	Chunking strategy with written justification	6
	Embedding model choice with written justification	4
	Retrieval with metadata filtering	3
	RAG prompt enforcing grounding	2
MCP Server (10)		
	Runs in its own container	3
	Exposes 2+ working tools	4
	Callable by agent system	3
Docker (10)		
	docker-compose.yml with all services	4
	Each component in its own container	3
	System starts with docker-compose up and works	3
API Layer (10)		
	FastAPI with POST /chat endpoint	3
	Streaming support	3
	Session management	2

	Error handling and timeouts	2
Guardrails (5)		
	Input and Output guardrail	3
	Graceful error handling and Iteration limits on agent loops	2
Evaluation (15)		
	Test set: 20+ questions with ground truth and sources	4
	Retrieval metrics (P@K, R@K, MRR) computed	3
	Generation evaluation on 20+ questions	3
	2+ configuration comparisons with metrics	3
	3 documented failure cases with root cause	2
Documentation (10)		
	Clean, well-commented code	2
	README with setup, architecture diagram, decisions, reasoning, results, changes tracking.	8
Bonus: Voice (+10)		
	STT input (Whisper)	1
	TTS output	1
	End-to-end voice demo	8
	Bert Finetuning	5
TOTAL		100 + 10 + 5

4. What We Are Really Evaluating

The rubric measures points, but here is what we are actually looking for:

- **1. Architecture understanding:** Can you design a system where independent services communicate? If Agent B is just a function call inside Agent A, you missed the point.

- **2. RAG depth:** Not just 'it retrieves something.' Did you think about chunk size? Did you measure retrieval quality? Did you iterate based on metrics?
- **3. Agent design:** Did you decompose the problem into meaningful specialist roles? Are the system prompts well-crafted? Does the supervisor make good routing decisions?
- **4. Evaluation maturity:** This is the clearest signal of real understanding vs. AI-generated code.
- **5. Production thinking:** Guardrails, error handling, session management, timeouts. These details show you understand what production means.

5. Deliverables

5.1 GitHub Repository (or zip file)

- All source code, organized in clear folder structure
- docker-compose.yml at the root
- Dockerfile for each service
- requirements.txt for each service
- .env.example (with placeholder keys, NEVER commit real keys)

5.2 README.md

- Project description (what it does, who it is for)
- Architecture diagram (ASCII, Mermaid, or image)
- Setup instructions (how to run with docker-compose)
- Technical decisions and justifications
- Known limitations

5.3 Evaluation Report (EVALUATION.md or notebook)

- Test set (20+ questions with ground truth)
- Retrieval metrics table
- Generation evaluation scores
- Configuration comparison results
- 3 failure cases with analysis

5.4 Demo (live + recorded for cases with technical issues)

- 5-minute walkthrough showing the system working
- Show at least 5 queries covering different agent routes
- Show one failure case and explain it
- Show the Docker containers running

6. Timeline

Date	Milestone
Mar 19	Chosen topic submission – required
Mar 22	To submit initial architecture plan (1-page: what you build/will build, containers, tools) - required
Mar 24*	1-1 Q/A and Follow up - required
Mar 27	FINAL SUBMISSION DEADLINE + Presentations

Note on AI Coding Assistants

We understand that AI coding assistants (Claude Code, Cursor, GitHub Copilot) can generate much of this code.

However, the following will be evaluated closely:

1. **Architecture decisions and justifications:** Why did you choose this chunk size? Why this model? Why this routing logic? You must explain and defend choices in person during the presentation.
2. **Evaluation and failure analysis:** Explaining WHY a specific query failed, WHICH component caused it, and WHAT you tried to fix it requires genuine understanding.
3. **Configuration comparisons:** Showing you experimented with different settings, measured results, and drew conclusions demonstrates real engineering judgment.

If your code works perfectly but you cannot explain how it works, that is a problem. If your code has issues but you can clearly explain what went wrong and what you would do differently, that demonstrates real learning.